ZineNow — Context & Handoff

Read this first when resuming on another machine. It covers what the project is, how it's built, what was done, what's pending, and the testing gotchas that cost the most time.

Last updated: 2026-09-19.

1. Project overview

ZineNow is a suite of zero-build, browser-only tools for making zines. Everything is static files — no npm, no bundler. Open via `file://` or any static server.

App	Path	Purpose
ZEditor	`apps/ZEditor/index.html` + `style.css`	Edit PDF pages: text, images, drawing, filters, page management. Saves editable `.znow` projects.
ZineArranger	`apps/ZineArranger/index.html`	Imposes a multi-page PDF into a printable zine booklet.
Md to PDF Converter	`apps/Markdown-to-PDF/`	Markdown → paginated PDF eBook.
Launcher	`index.html`, `app.js`, `style.css`, `bg.jpg` (root)	Landing page linking to the apps. (`app-logo.png` referenced by it is missing — pre-existing 404, harmless.)

2. Architecture notes per app

ZEditor (single ~3400-line file)

- React + Babel Standalone in one `<script type="text/babel">`. Local vendored `pdfjs/` (rendering) and `pdf-lib/` (export).
- **State is immutable everywhere** (`.map` / spread, never mutate). Undo snapshots rely on this for structural sharing — keep it that way.

- Page object: `{ key, docId, pageNumber, rotation, dataUrl, isBlank, annotations, filter }`. Blank pages have `width/height` instead of `docId/pageNumber`.
- `pdfDocsRef.current` = `{ docId: { doc, bytes } }` — multiple source PDFs can be loaded (Insert PDF). Use `docFor(pdfDocsRef, page)`.
- Undo/redo: call `recordUndoCheckpoint(coalesceKey?)` BEFORE a mutating `setPages`. Same key within `UNDO_COALESCE_MS` (700ms) merges into one step. Thumbnails (`dataUrl`) are stripped from snapshots and regenerated.
- Annotations: types `text`, `image`, `stroke`. Edited through `updateAnnotationsFor(pageIdx, updater, coalesceKey)`.
- **Filters (grayscale/duotone/halftone) are per-page** (`page.filter`, default `DEFAULT_PAGE_FILTER = {type:"none"}`), edited via `updatePageFilter(pageIdx, patch)`. Images have their own `ann.filter` (`match` = follow the page's filter). Both preview and export read `page.filter`.
- Export: `buildExportPdfBytes()` renders every page to canvas (`renderPageToCanvas`) and embeds PNGs via pdf-lib.
- Project file `.znow` = JSON (base64-embedded source PDFs + pages minus thumbnails + settings). Self-contained; thumbnails regenerated on load.
- Selection model: primary `selectedIdx` / `selectedAnnId` + secondary arrays `selectedPageIndices` / `selectedAnnIds` (Shift+Click multi-select).
- Keyboard shortcuts (window `keydown` effect): Delete, Space=Select, T, I, H, D=Draw, Ctrl+Z/Y, Ctrl+R (rotate), Ctrl+C/V (internal annotation copy/paste), ArrowUp/Down (page nav), Escape (leave text editing but keep selected). A separate `paste` listener handles pasting an image from the system clipboard.
- Page reorder in the sidebar uses native HTML5 drag-and-drop (`Sidebar` → `dragHandlersFor`); annotation dragging uses manual mouse listeners — different mechanisms.

Md to PDF Converter

- **`App.jsx` is precompiled plain JS — `React.createElement(...)`, NOT JSX.** There is no Babel here. Edit it as plain JS or the page silently breaks.
- Normal export (`exportToPdf` in `pdfGenerator.js`) uses the browser's native print dialog (`window.print()` in a hidden iframe). It never yields PDF bytes in JS — the code comments explain it deliberately avoids html2canvas.
- Live preview: `renderHtmlDocument()` in a `sandbox="allow-scripts"` iframe (opaque origin, on purpose) paginated by Paged.js; talks to the parent only via

`postMessage`.

- Depends on CDNs (marked, mammoth, turndown, jszip, pdf-lib, DOMPurify, Tailwind, React, Google Fonts, Paged.js, html2canvas) — needs internet.

ZineArranger

- Large single file. Has a "ZEditor handoff" block near the end: listens for `postMessage` `{type:"zeditor-handoff-pdf", buffer, filename}` and feeds the bytes into its `#selector` file input; pings `{type:"zinearranger-ready"}` to `window.opener` every 500ms (up to 15×).

Cross-app handoff protocol ("Send to ZineArranger")

Sender (ZEditor and Md-to-PDF both use the same shape):

1. `window.open("../ZineArranger/index.html","_blank")` synchronously in the click handler (popup-blocker safe).
2. Attach a `message` listener **immediately** (before any `await`) for `zinearranger-ready`.
3. Build PDF bytes, await ready, then `win.postMessage({type:"zeditor-handoff-pdf", buffer, filename}, "*", [buffer])`. The message type is named after ZEditor but ZineArranger doesn't check the sender, so Md-to-PDF reuses it as-is.

3. What was done in this project's recent sessions

ZEditor:

- Fixed Send-to-ZineArranger race (listener registered too late) + repeated ready pings.
- Save/Load Project (`.znow`), Insert PDF as pages (with "insert after page" dialog), New Project / Load Project buttons with two-step erase confirmation.
- Text box: removed 320px max width, no shrink-on-blur, free resize.
- Per-image filters; halftone slider min 1px.
- Keyboard shortcuts + Shift+Click multi-select (highlight + group Delete only).
- **Page filter is now per-page** (was one global filter applied to all pages, including export).
- **Ctrl+V pastes an image from the system clipboard** (`addImageAnnotationFromFile`, shared with the Image tool's file picker).

Md to PDF Converter:

- **"Send to ZineArranger" button** (`handleSendToZineArranger` in `App.jsx`; `renderRasterizationDocument` in `pdfGenerator.js`). Pipeline: hidden iframe → Paged.js paginates → html2canvas rasterizes each `.pagedjs_page` at scale 2 → images `postMessage`d back → pdf-lib assembles a PDF → handoff to ZineArranger.
- Security decision: html2canvas **hangs forever** inside a `sandbox="allow-scripts"`-only iframe, so the export iframe uses `allow-scripts allow-same-origin`. That combo would let a `<script>` smuggled via pasted Markdown raw HTML reach the parent app, so the rendered HTML is run through **DOMPurify first**. Do not remove the sanitize step or the reasoning comments. The live preview iframe stays opaque-origin and untouched.

4. Pending / known gaps (priority order)

1. **Group drag-reorder of multi-selected pages — NOT implemented.** Request: when several pages are Shift-selected and one is dragged in the sidebar, the whole selected set moves together to the drop position. Plan: in `Sidebar.dragHandlersFor(idx).onDrop`, if the dragged page is part of a multi-selection, call a new `handleReorderGroup(sortedFromIndices, toIdx)` in `App` (pattern: `recordUndoCheckpoint()`, immutable `setPages` that removes all selected as a block preserving relative order and reinserts at the adjusted target, then recompute `selectedIdx`/`selectedPageIndices`). Needs a new `onReorderGroup` prop or reuse `onReorder`. Existing single-page `handleReorder` is at ~line 2846 of ZEditor's `index.html`.
2. **Md-to-PDF "Send to ZineArranger" is not verified end-to-end through the real handoff.** The rasterize → PDF pipeline and the sanitization were verified in isolation; the actual `window.open` + postMessage round trip could not be exercised in the test browser (see §5). Test it once in a normal Chrome/Edge window.
3. Md-to-PDF rasterization with Markdown images (`![](url)`): cross-origin images could taint the canvas (`useCORS: true` is set, untested with remote images).
4. Group drag/resize of multi-selected **annotations** was deliberately cut (only highlight + group Delete work).
5. Old `.znow` files saved before per-page filters stored one global filter in `settings`; that is now ignored on load (those projects open with no filters).
6. Sidebar thumbnails don't reflect filters (only the main canvas and export do).

7. Edge case: if an annotation was copied in-app (Ctrl+C) AND the OS clipboard also holds an image, Ctrl+V does both (duplicate + image paste).
8. Md-to-PDF asks a ~3-page document to paginate twice (preview + export iframe) and Paged.js/html2canvas are slow; a progress indicator on the button would be a nice touch.

5. Testing setup and gotchas (read before debugging "it's stuck")

Dev server: any static server rooted at the project folder, e.g. `python -m http.server 8951 --directory "<path to ZineNow>"`, then open `http://localhost:8951/apps/ZEditor/index.html`. Under Claude Code's preview tooling this is done via a `.claude/launch.json` in the session's primary working directory (not necessarily the project folder). Remove it when done.

Things that looked like bugs but weren't:

- **Hidden browser pane throttles** `requestAnimationFrame` → pdf.js rendering, Paged.js and html2canvas appear to hang. Taking a screenshot foregrounds the pane briefly; it reverts to hidden between tool calls. Poll with screenshot + wait inside one `browser_batch`, and keep patience (Paged.js + html2canvas take ~10–30s here).
- **HTTP cache is sticky**: after editing `pdfGenerator.js` etc., a plain reload kept serving the stale script, even in a new tab. Use a **new port** (new origin) or a cache-busting query on the page; note sub-resources (`<script src="pdfGenerator.js">`) do not inherit the page's query string. Verify with `fn.toString().includes("...")`.
- `window.open(_, "_blank")` **in the test pane replaces the current tab**, destroying the sender before it can finish the handoff. Not representative of a real browser.
- The network-request tool only logs same-origin requests; CDN loads never appear. Console history can persist across navigations.
- Loading a PDF into ZEditor without a file dialog: build a `File`, put it in a `DataTransfer`, assign to the hidden `input[type=file][accept="application/pdf"]`, dispatch `input` + `change`. A tiny test PDF can be made in-page with `PDFLib`.
- Capturing Export/Save output: set `window.showSaveFilePicker = undefined`, monkey-patch `URL.createObjectURL` to stash the Blob, click the button, then inspect the blob (e.g. render pages with `pdfjsLib` and sample pixels).

- ZEditor helper functions such as `renderPageToCanvas` and `applyPageFilter` are top-level and reachable from the console; anything inside `App()` is not.

6. Resuming on a new machine

1. Copy the whole `ZineNow` folder (this file included). It is plain files; no install step.
2. Claude Code's auto-memory and session transcripts are local to the old machine and do **not** transfer — this file is the substitute. Consider renaming/copying it to `CLAUDE.md` in the project root so a new session loads it automatically.
3. Start with pending item #1 (group page drag) unless priorities changed, then item #2 (real-browser test of the Md-to-PDF handoff).
4. Working conventions carried over from the previous sessions: keep edits inside the existing single-file structure, preserve immutable state updates in ZEditor, default to no code comments unless the *why* is non-obvious, and test UI changes in a browser before calling them done.